Rubrica para avaliação de trabalhos			
Disciplinas	**Sprint**	**Descrição**	**Nota Maxima**
Back-end com IA Generativa	☐	Autenticação - Login com e-mail e senha - Senha criptografada no banco - Validação de e-mail (regex) - Retorno JWT - Logar apenas usuário existente	0,5
	☐	Cadastro de Usuário - Cadastro com os campos obrigatórios: nome, email, senha, CPF - Validação de CPF - Validação de nível de senha - Validação de email	1
	☐	Edição de Usuário - Rota autenticada - Só pode editar o próprio usuário - Garantir que todos os campos sáo obrigatórios - Validação de CPF - Validação de nível de senha - Não permitir que o usuário altere o email	1
	☐	Cruds Completos: - 4 CRUDS completos (Exceto o CRUD de usuários) - 4 CRUDS com todas as rotas autenticadas - Paginação das listagens - Não editar/deletar recursos que não existem - Ao menos 1 relacionamento entre os recursos	1
	☐	Configuração de Agentes: - Criação do arquivo AGENTS.md - Definição de 3 skills específicas para os agentes - Implementação de guardrails para limitar ações e comportamentos do agente - Proteção de arquivos sensíveis, impedindo que o agente leia ou exponha variáveis e credenciais do .env	0,5
Front-end	☐	Autenticação - Campos para email e senha - Validação de email (regex) - Armazenar o token no localstorage - Lidar com os erros da API de forma amigável para o usuário - Redirecionar pra área logada	0,50
	☐	Cadastro de Usuário - Campos obrigatórios: nome, email, senha, CPF - Verificação dupla de senha - Validação de CPF - Validação de email - Redirecionar para o login após o cadastro - Lidar com as mensagens de erro da API	1,00
	☐	Edição de Usuário - Utilizar contexto para exibir informações do usuário globalmente - Validação de CPF - Validação de nível de senha - Confirmação da senha - Não permitir que o usuário altere o email	1,00
	☐	Cruds Completos - 4 CRUDS completos (Exceto o CRUD de usuários) - Paginação das listagens - Validação dos campos obrigatórios - Paginação e cadastro/edição em telas separadas	1,00
	☐	Componentização - Componentes genéricos e reutilizáveis - Telas compostas por vários componentes - Separação dos componentes e páginas em pastas separadas - Componentes exclusivos da página em funções separadas	0,50
Ferramentas DevOps	☐	Organização do docker-compose.yml (Código bem indentado, com separação clara de serviços, uso de volumes, redes e variáveis de ambiente.)	0,50
	☐	Integração entre serviços (Backend conecta ao MySQL, frontend consome o backend, Nginx atua como proxy.)	0,50
	☐	Persistência de dados no MySQL (Uso correto de volumes para manter os dados mesmo após reinício.)	0,50
	☐	Configuração do Nginx como proxy reverso (Redirecionamento de requisições para frontend e backend de forma funcional.)	1,50
	☐	Isolamento e restrição de acesso via Docker Network: Garantir que apenas o Nginx esteja exposto diretamente fora do container.	1,00
	☐		0,50

Tech Forge	☐	**Encapsulamento e Proteção do Estado**, para avaliar se há proteção do estado interno dos objetos contra modificações indevidas, observando o uso correto de modificadores de acesso (private, protected, public) ou campos privados nativos (#field), além de construtores limpos com a devida inicialização das propriedades.	0,50
	☐	**Acesso Controlado e Propriedades**, para avaliar se a exposição de dados através de getters e setters (get / set) é feita apenas quando existem validações, transformações ou regras de negócio necessárias, evitando expor atributos de forma desnecessária.	0,50
	☐	**Modelagem do Domínio e Pilares da POO**, para avaliar se a abstração do problema em entidades bem estruturadas foi realizada corretamente, observando o uso de classes e atributos para representar o sistema, a aplicação adequada de herança e classes abstratas (abstract class) onde há reuso legítimo de código, e a presença de polimorfismo através de contratos comuns ou sobrescrita de métodos.	0,50
	☐	**Tipagem Avançada e Recursos do TypeScript**, para avaliar se há o aproveitamento dos recursos nativos da linguagem, observando a definição de estruturas com interfaces e type aliases, a ausência do uso indevido de any para garantir a segurança dos tipos, e a aplicação de Generics para o reuso de código tipado.	0,50
	☐	**Aderência aos Princípios SOLID**, para avaliar se a qualidade do design orientado a objetos atinge coesão e desacoplamento, observando a presença de classes com responsabilidade única (S), a capacidade de extensão sem alteração do código base (O) e a garantia de que as subclasses possam substituir as classes base sem quebrar a aplicação (L).	0,50
	☐	**Tratamento de Exceções e Organização do Código**, para avaliar se a robustez e a clareza da estrutura do projeto foram mantidas, observando o lançamento de erros customizados através de classes de exceção, a distribuição organizada dos arquivos em pastas (entidades, serviços, repositórios) e a padronização de nomenclatura segundo as convenções da linguagem.	0,50